

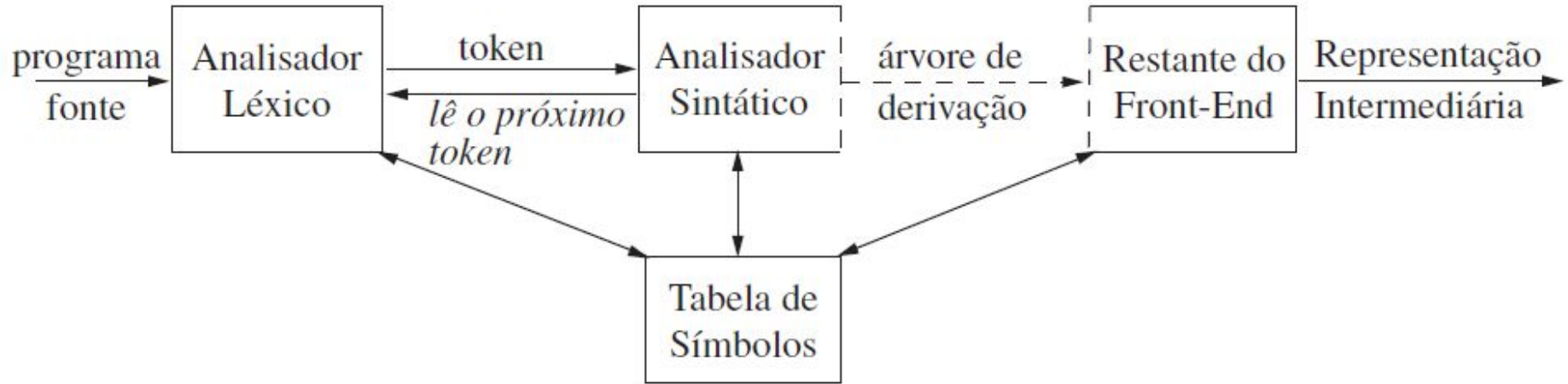
INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
CEARÁ
CAMPUS ARACATI

Bacharelado em Ciência da Computação

Disciplina: Compiladores
Aula 03 - Análise Sintática

Professor: Diego Rocha Lima

Analizador Léxico → Analizador Sintático → Analizador Semântico



O analisador sintático recebe do analisador léxico uma cadeia de tokens representando o programa fonte verifica se essa cadeia de tokens pertence à linguagem gerada pela gramática.

Análise Sintática

- O analisador deve ser projetado para emitir mensagens para quaisquer erros de sintaxe encontrados no programa fonte, mas continuar processando o restante do programa.
- O analisador sintático constrói uma árvore de derivação e a passa ao restante do front-end do compilador para mais processamento.
- Assim, o analisador sintático e o restante do front-end podem ser implementados em um único módulo.
- Existem três estratégias gerais de análise sintática para o processamento de gramáticas: universal, descendente e ascendente.

Análise Ascendente vs Descendente

- **Análises descendentes** constroem as árvores de derivação de cima (raiz) para baixo (folhas),
- Métodos **ascendentes** fazem a análise no sentido inverso, começam nas folhas e avançam até a raiz construindo a árvore.
- Em ambas as estratégias, a entrada do analisador sintático é consumida da esquerda para a direita, um símbolo de cada vez.

Gramáticas LL vs LR

A Análise Sintática verifica se a sequência de tokens forma uma estrutura válida segundo a gramática.

- Top-down (LL): começa do símbolo inicial e tenta gerar a entrada.
- Bottom-up (LR): parte da entrada e tenta reduzir até o símbolo inicial.

LL(1)

- L = **Left-to-right** leitura da entrada
- L = **Leftmost** derivation (derivação mais à esquerda)
- 1 = Usa 1 token de lookahead

LR(0)

- L = **Left-to-right** leitura da entrada
- R = **Rightmost** derivation (in reverse) (reduz da direita para a esquerda)
- 0 = Usa 0 tokens de lookahead

Gramáticas LL vs LR - Exemplos

Gramática LL(1)

A escolha entre as produções de **S** depende **apenas do primeiro token da entrada**:

$$S \rightarrow a A \mid b B$$
$$A \rightarrow c$$
$$B \rightarrow d$$

Ex.: ac,bd

Gramática LR(0)

Lê os símbolos da entrada (da esquerda pra direita), vai colocando na **pilha**. Tenta fazer **reduções**, substituindo símbolos por regras da gramática — **sem olhar o próximo token** (por isso o "0").

$$S \rightarrow A$$
$$A \rightarrow a A \mid b$$

Ex.: b,ab,aab,aaab...

Relembrando Gramáticas Regulares e Derivação

A linguagem $(a + b)^*(aa + bb)$ é gerada pelas seguintes gramáticas regulares:

a *Linear à direita.* $G = (\{ S, A \}, \{ a, b \}, P, S)$, e P possui as seguintes regras de produção:

$$\begin{aligned} S &\rightarrow aS \mid bS \mid A \\ A &\rightarrow aa \mid bb \end{aligned}$$

b *Linear à esquerda.* $G = (\{ S, A \}, \{ a, b \}, P, S)$, e P possui as seguintes regras de produção:

$$\begin{aligned} S &\rightarrow Aaa \mid Abb \\ A &\rightarrow Aa \mid Ab \mid \varepsilon \end{aligned}$$



- S e A são nós não terminais
- a e b são símbolos terminais
- P são as regras de produção
- S é o nó inicial para derivação

Relembrando Gramáticas Livres de Contexto

As **gramáticas livres de contexto** (ou **CFG – Context-Free Grammar**) são um tipo de gramática formal usada para definir **linguagens de programação, linguagens naturais, sintaxe de expressões**, entre outros.

Elas são chamadas “**livres de contexto**” porque as regras de substituição **não dependem do contexto onde o símbolo aparece** — só do **não-terminal** sendo substituído.

Símbolo	Significado
V	Conjunto de variáveis (não-terminais)
Σ	Alfabeto (conjunto de terminais)
P	Conjunto de produções
S	Símbolo inicial

$$G = (V, \Sigma, P, S)$$

$$V = \{S\}$$

$$\Sigma = \{a, b\}$$

$$P = \{ \\ S \rightarrow aSb, \\ S \rightarrow \epsilon$$

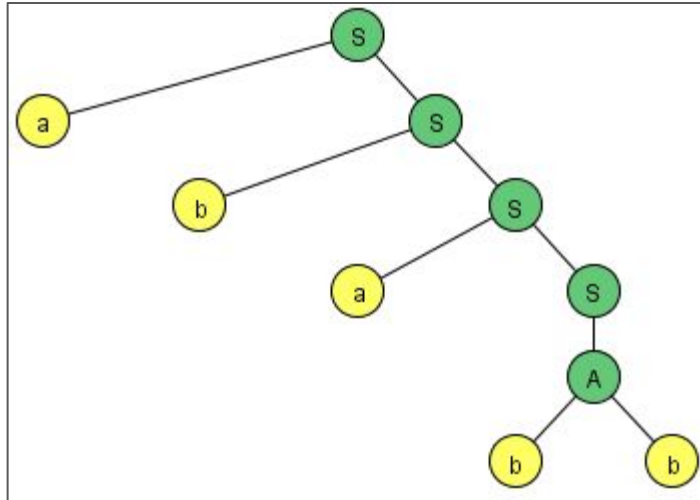
$$\}$$

$$S = S$$

Tipos de derivação

- Ex.: Com base na gramática G, derivar a cadeia **ababb**

LHS		RHS
S	$\rightarrow$	aS
S	$\rightarrow$	bS
S	$\rightarrow$	A
A	$\rightarrow$	aa
A	$\rightarrow$	bb



	S
$S \rightarrow aS$	aS
$S \rightarrow bS$	abS
$S \rightarrow aS$	abaS
$S \rightarrow A$	abaA
$A \rightarrow bb$	ababb

Representando Gramáticas

$$\begin{array}{lcl} E & \rightarrow & E + T \quad | \quad T \\ T & \rightarrow & T * F \quad | \quad F \\ F & \rightarrow & (E) \quad | \quad \mathbf{id} \end{array}$$

- **E, T e F** são nós não terminais
- **+, *, (,), id** são nós terminais
- Pertence a classe LR

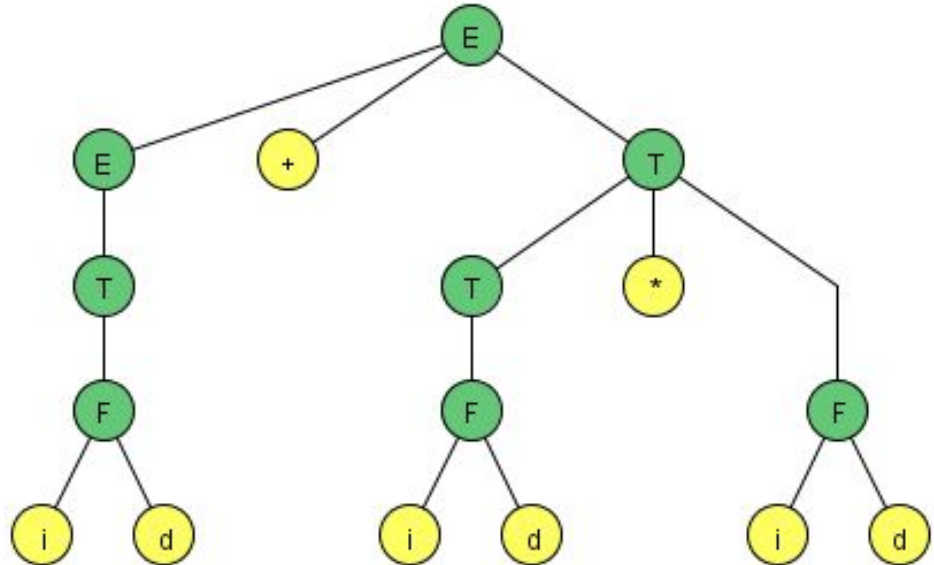
- O não-terminal E representa expressões consistindo em termos separados pelo operador +
- T representa termos consistindo em fatores separados pelo operador *
- F representa fatores que podem ser expressões entre parênteses ou identificadores.

Exemplos (derivação)

$E \rightarrow E + T$		T
$T \rightarrow T * F$		F
$F \rightarrow (E)$		id

- Obs: **id's** são tokens definidos na análise léxica

Derivar: **id + id * id**



Variações de Gramática

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow * F T' \mid \epsilon$$

$$f \rightarrow (E) \mid \mathbf{id}$$

$$E \rightarrow E + E \mid E * E \mid (E) \mid \mathbf{id}$$

- trata + e * da mesma forma, permitindo ambiguidades e mais de uma árvore de derivação

- sem recursão a esquerda
- permite apenas uma árvore de derivação, sem ambiguidades

Definindo a ordem das produções

Terminais: `id` , `+` , `-` , `*` , `/` , `(` , `)`

<i>expression</i>	$\rightarrow$	<i>expression</i> + <i>term</i>
<i>expression</i>	$\rightarrow$	<i>expression</i> - <i>term</i>
<i>expression</i>	$\rightarrow$	<i>term</i>
<i>term</i>	$\rightarrow$	<i>term</i> * <i>factor</i>
<i>term</i>	$\rightarrow$	<i>term</i> / <i>factor</i>
<i>term</i>	$\rightarrow$	<i>factor</i>
<i>factor</i>	$\rightarrow$	(<i>expression</i>)
<i>factor</i>	$\rightarrow$	id

Monte a árvore de derivação para:

- `id + id * id`
- `id * id + id`



<i>E</i>	$\rightarrow$	<i>E</i> + <i>T</i> <i>E</i> - <i>T</i> <i>T</i>
<i>T</i>	$\rightarrow$	<i>T</i> * <i>F</i> <i>T</i> / <i>F</i> <i>F</i>
<i>F</i>	$\rightarrow$	(<i>E</i>) id

Mais exemplos de derivações

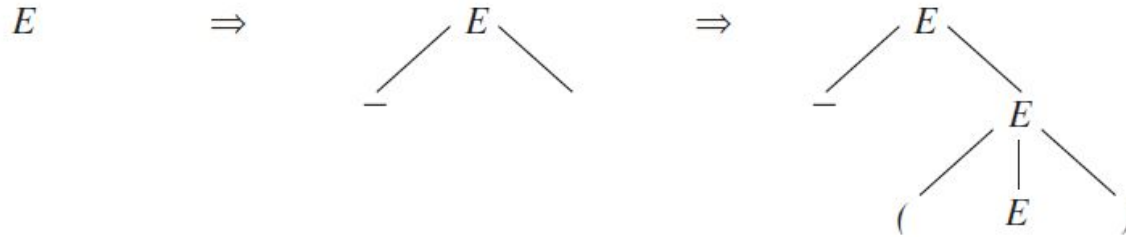
$$E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid \mathbf{id}$$

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(\mathbf{id})$$

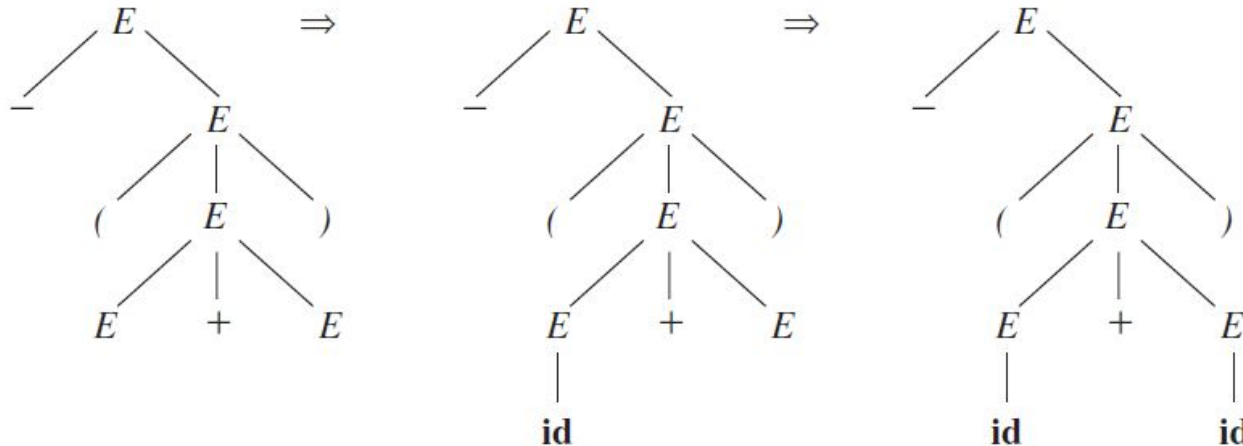
$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\mathbf{id} + E) \Rightarrow -(\mathbf{id} + \mathbf{id})$$

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(E + \mathbf{id}) \Rightarrow -(\mathbf{id} + \mathbf{id})$$

Mais exemplos de derivações

$$E \rightarrow E + E \mid E * E \mid -E \mid (E) \mid \text{id}$$


- (id + id)



Ambiguidades

permite duas derivações mais à esquerda distintas para a sentença: **id + id * id**:

$$\begin{aligned} E &\Rightarrow E + E \\ &\Rightarrow \mathbf{id} + E \\ &\Rightarrow \mathbf{id} + E * E \\ &\Rightarrow \mathbf{id} + \mathbf{id} * E \\ &\Rightarrow \mathbf{id} + \mathbf{id} * \mathbf{id} \end{aligned}$$

$$\begin{aligned} E &\Rightarrow E * E \\ &\Rightarrow E + E * E \\ &\Rightarrow \mathbf{id} + E * E \\ &\Rightarrow \mathbf{id} + \mathbf{id} * E \\ &\Rightarrow \mathbf{id} + \mathbf{id} * \mathbf{id} \end{aligned}$$

Escrevendo Gramáticas

Para que as seu analisador sintático seja mais eficiente, algumas operações devem ser feitas a fim de evitar erros no analisador:

1. Eliminar ambiguidades
2. Eliminar recursão à esquerda
3. Fatoração à esquerda

Eliminação de Ambiguidades

stmt → **if** *expr* **then** *stmt*
| **if** *expr* **then** *stmt* **else** *stmt*
| **other**

Gramática Ambígua

Alternativa não ambígua

stmt → *matched_stmt*
| *open_stmt*
matched_stmt → **if** *expr* **then** *matched_stmt* **else** *matched_stmt*
| **other**
open_stmt → **if** *expr* **then** *stmt*
| **if** *expr* **then** *matched_stmt* **else** *open_stmt*

A eliminação de ambiguidades evita erros na etapa de análise sintática

Eliminação de recursão à esquerda

$$\begin{array}{lcl} E & \rightarrow & E + T \mid T \\ T & \rightarrow & T * F \mid F \\ F & \rightarrow & (E) \mid \mathbf{id} \end{array}$$

Com recursão
à esquerda

Os métodos de análise descendentes não podem tratar gramáticas com recursão à esquerda, de modo que uma transformação é necessária para eliminar essa recursão

Sem recursão
à esquerda

$$\begin{array}{lcl} E & \rightarrow & TE' \\ E' & \rightarrow & +TE' \mid \epsilon \\ T & \rightarrow & FT' \\ T' & \rightarrow & *FT' \mid \epsilon \\ F & \rightarrow & (E) \mid \mathbf{id} \end{array}$$

Fatoração à esquerda

Quando a escolha entre duas ou mais alternativas das produções-A não é clara, ou seja, elas começam com a mesma forma sentencial, podemos reescrever essas produções para adiar a decisão até que tenhamos lido uma cadeia da entrada longa o suficiente para tomarmos a decisão correta.

$$\begin{array}{l} S \rightarrow iEtS \mid iEtSeS \mid a \\ E \rightarrow b \end{array}$$
$$\begin{array}{l} S \rightarrow iEtSS' \mid a \\ S' \rightarrow eS \mid \epsilon \\ E \rightarrow b \end{array}$$

Com a criação do S', adiamos a decisão de derivar um “else” ou não após o “if”

Exercício

1 - Criar um analisador Léxico e Sintático no GALS capaz de reconhecer expressões matemáticas e simular uma calculadora

Obs.: A sintaxe utilizada no GALS está no material de estudo